



Java Collections Framework

Summary

The Java Collections Framework is a set of interfaces and classes that represent groups of objects as a single unit. It provides highly efficient, reusable data structures.

The framework is divided into two main hierarchies: **Collection** and **Map**.

1. The Core Collection Interfaces

The Collection interface is the root of the hierarchy (excluding Map). It provides a common set of methods (add(), remove(), size()) that are inherited by three primary sub-interfaces:

Interface	Key Characteristics	Typical Implementations	Best Used When...
List	Ordered collection (maintains insertion order). Allows duplicate elements.	ArrayList, LinkedList, Vector	You need elements stored by index and can have duplicates (e.g., a list of transaction items).
Set	Unordered collection. Prohibits duplicate elements.	HashSet, LinkedHashSet, TreeSet	You need to store unique items and fast lookups are important (e.g., a list of unique student IDs).
Queue	Primarily for holding elements before processing. Typically ordered by FIFO (First-In, First-Out).	PriorityQueue, ArrayDeque	You need a structure to manage tasks in the order they were submitted (e.g., a print job queue).

Example: Choosing between List and Set

```
import java.util.ArrayList;  
import java.util.HashSet;
```

```

public class CollectionChoice {
    public static void main(String[] args) {
        // LIST: Allows duplicates and keeps order
        ArrayList<String> shoppingList = new ArrayList<>();
        shoppingList.add("Milk");
        shoppingList.add("Milk"); // Duplicate is allowed
        System.out.println("List: " + shoppingList); // Output: [Milk, Milk]

        // SET: Removes duplicates, order is not guaranteed
        HashSet<String> uniqueItems = new HashSet<>();
        uniqueItems.add("Apple");
        uniqueItems.add("Apple"); // Duplicate is ignored
        System.out.println("Set: " + uniqueItems); // Output: [Apple]
    }
}

```

2. The Map Interface (Key-Value Pairs)

The Map interface is **not** a true Collection (it doesn't inherit from the Collection interface), but it is a vital part of the framework. It stores data as **Key-Value pairs**.

Key Characteristics

- **Keys must be unique:** You cannot have duplicate keys.
- **Values can be duplicated.**
- The key is used to quickly retrieve the associated value.

Interface	Key Characteristics	Typical Implementations	Best Used When...
Map	Stores (Key, Value) pairs. Keys are unique.	HashMap, LinkedHashMap, TreeMap	You need to associate one piece of data with another (e.g., mapping usernames to passwords).

Map Usage Example

```
import java.util.HashMap;
```

```
public class MapExample {  
    public static void main(String[] args) {  
        // Maps String keys to Integer values  
        HashMap<String, Integer> scores = new HashMap<>();  
  
        // 1. Add key-value pairs (uses .put())  
        scores.put("Alex", 95);  
        scores.put("Mia", 88);  
  
        // 2. Retrieve a value by key (uses .get())  
        int alexsScore = scores.get("Alex"); // Returns 95  
        System.out.println("Alex's Score: " + alexsScore);  
  
        // 3. Keys must be unique (This overwrites Alex's original score)  
        scores.put("Alex", 98);  
        System.out.println("Updated Score: " + scores.get("Alex")); // Returns 98  
    }  
}
```